MODULE 16 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Getting Data into BigQuery

Load jobs, the Storage Write API, subscriptions, change data capture, external tables, and deduplication.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 16 of 45

admin@sthithapragna.com

Batch is free and patient. Streaming is fast and billed. **External tables do not move the data at all.**

Six ways into BigQuery

METHOD	BEST FOR	COST AND FRESHNESS
Load jobs	Batches of files	Free in the shared pool
Storage Write API	Streaming from code or Dataflow	Seconds; 2 TiB a month free
Pub/Sub subscription	Messages straight to a table	Seconds; no code
Datastream	Database changes	Minutes; managed CDC
Transfers, scheduled queries	SaaS, warehouses, recurring SQL	On a schedule
External tables	Files queried where they sit	No load; slower queries

THE TIEBREAKER

Queried many times a day: load it. Queried rarely: it can stay outside.

Load jobs: the free, patient door

- Avro, Parquet, ORC, CSV and JSON. **Free** in the shared slot pool.
- **Autodetect** infers a schema, but misses headers when every column is text.
- **max_bad_records** defaults to **0**: one bad row fails the load.
- Gzip CSV cannot be read in parallel, and is limited to 4 GB per file. Append or overwrite.

> THE TIP
>
> Self-describing formats like Avro and Parquet carry their own schema, so autodetect is not needed.

Storage Write API or legacy streaming?

STORAGE WRITE API, GRPC

The recommended way

- Exactly-once with committed streams
- Atomic batch commits with pending streams
- USD 0.025 per GiB, 2 TiB a month free
- Google's choice for new projects

LEGACY STREAMING, REST

Formerly insertAll

- Now called Storage Write API (REST)
- insertId: best-effort dedupe, one minute
- USD 0.01 per 200 MiB, 1 KB minimum per row
- Still supported, not recommended

CURRENCY CHECK

'Streaming inserts' in older questions means insertAll. The modern answer is the Storage Write API.

Four kinds of write stream

STREAM	SEMANTICS	USE IT FOR
`Default`	At least once, visible at once	High-throughput streaming, duplicates tolerated
`Committed, with offsets`	Exactly once, visible at once	Streams where every row must land once
`Pending`	Invisible until an atomic commit	Batch loads that must land all or nothing
`Buffered`	Held until flushed	Advanced cases, mostly inside Beam

THE SIGNAL

'No duplicates, written from our own code' means committed streams with offsets.

Streaming without writing the client

- **Pub/Sub BigQuery subscription**: messages straight into a table, no code. At least once.
- **Dataflow BigQueryIO** uses the Storage Write API, when messages need processing first.
- **Continuous queries** stream results out of BigQuery, into Pub/Sub, Bigtable or tables.
- All of them land rows in seconds. They differ in where the logic lives.

> THE TIEBREAKER
>
> No transformation: subscription. Transformation or joins: Dataflow. Logic in SQL on data already in BigQuery: continuous queries.

Three ways to apply changes

- **Datastream** from a database: managed CDC, merge or append-only. Module 08.
- **CDC through the Storage Write API**: tag rows UPSERT or DELETE, into a table with a primary key.
- Primary keys are **not enforced**, and active CDC tables cannot be changed with DML.
- Or land changes **append-only**, then apply them with a scheduled **MERGE**.

> REMEMBER
>
> max_staleness trades freshness for cost. Background apply jobs use a BACKGROUND reservation, or on-demand.

The classic MERGE, with deduplication

```
1  MERGE curated.accounts AS t
2  USING (
3    SELECT * FROM raw.account_changes
4    WHERE change_ts > @last_run
5    QUALIFY ROW_NUMBER() OVER (
6      PARTITION BY account_id ORDER BY change_ts DESC) = 1
7  ) AS s
8  ON t.account_id = s.account_id
9  WHEN MATCHED AND s.op = 'D' THEN DELETE
10 WHEN MATCHED THEN UPDATE SET tariff_id = s.tariff_id, iban_token = s.iban_token
11 WHEN NOT MATCHED AND s.op != 'D' THEN
12   INSERT (account_id, customer_id, tariff_id, iban_token)
13   VALUES (s.account_id, s.customer_id, s.tariff_id, s.iban_token)
```

5-6 Keep only the latest change per account, or MERGE meets two matches.

9-10 Deletes remove the row. Updates overwrite it.

THE PATTERN

Rerunning this MERGE gives the same result. Idempotent loads make retries safe.

External tables, or load the data?

QUERY IN PLACE

External and Lakehouse tables

- No load step, files stay put
- Always as fresh as the files
- Slower, with fewer features
- Federated queries reach Cloud SQL, Spanner, AlloyDB

LOAD

Native BigQuery tables

- A load or stream step
- Fresh as of the last load
- Fastest queries, every feature
- Partitioning, clustering, caching

THE TIEBREAKER

'Query occasionally, keep one copy' points to external. 'Dashboards hit it all day' points to loading.

Data Transfer Service and scheduled queries

- **Data Transfer Service**: scheduled loads from SaaS apps, warehouses, S3, Azure and Cloud Storage.
- **Scheduled queries**: one SQL statement on a timetable, writing a destination table.
- Need steps in order, tests and version control? That is **Dataform**.
- Need other services in the chain? That is **Managed Airflow** or Workflows. Module 35.

> THE TIEBREAKER
>
> One query, no dependencies: a scheduled query. A graph of SQL: Dataform.

Deduplicating Larkspur's readings

- Meters **resend** batches, so raw readings contain exact and near duplicates.
- Keep one row per **meter_id** and **reading_ts**, the latest by **received_ts**.
- **SELECT DISTINCT** fails, because each copy has a different received time.
- Neither insertId nor exactly-once delivery covers resends from the source.

> THE PATTERN
>
> Deduplicate on the business key, in a step you can rerun. Never rely on the transport.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Nightly files, cheapest	Load jobs
Exactly once from our own code	Storage Write API, committed stream
Streaming inserts, insertAll	Legacy; prefer the Storage Write API
Upserts and deletes, no MERGE jobs	CDC with UPSERT and DELETE
Queried **all day**, stored as files	Load into native tables
One latest row per key	QUALIFY ROW_NUMBER, then MERGE

THE PATTERN

Freshness decides the door. Query frequency decides whether to load at all.

> Every night, Larkspur's billing system exports about 200 GB of Parquet files to Cloud Storage. Analysts query the results all the next day.
>
> Finance wants the cheapest reliable way to get them into BigQuery by 06:00.

How should the files be ingested?

A. A load job from Cloud Storage into a native table, after each export

B. Legacy streaming inserts, sending every row as soon as it is read

C. A Dataflow pipeline that streams the rows in with the Storage Write API

D. External tables over the files, queried directly by every analyst

◆ PAUSE AND DECIDE

How should the files be ingested?

A A load job

Free in the shared pool, batch-shaped, and the native table is fast all day.

B Legacy streaming

Billed per row, for data that arrives once a night anyway.

C Dataflow streaming

A pipeline and per-GiB charges, for a nightly batch.

D External tables

Free to set up, but slow and costly when queried all day.

THE SIGNAL

Nightly files, cheapest, queried all day: a load job into a native table.

> Larkspur's payment service is written in Java and writes each payment event straight into BigQuery.
>
> After network errors it retries, and finance has found duplicate payments in the table. Every payment must land exactly once.

How should the service write to BigQuery?

A Legacy streaming inserts, with an insertId on every row it sends

B The Storage Write API, using a committed stream with offsets

C The Storage Write API default stream, retrying any failed append

D A load job every minute, from files written by the service

◆ PAUSE AND DECIDE

How should the service write to BigQuery?

A insertId

Best-effort dedupe for about a minute. Google says not to rely on it.

B Committed stream, offsets

A repeated offset is rejected, so a retry cannot land twice.

C Default stream

At least once by design. Retries can still duplicate.

D A load per minute

A file pipeline, with load-job limits, to fix a write problem.

THE SIGNAL

'Exactly once' from your own code: a committed stream with offsets.

A third-party pricing engine at Larkspur emits a change event whenever a tariff is created, changed or withdrawn. It has no database that Larkspur can read.

A BigQuery table must mirror the current tariffs within minutes, and the team does not want to run MERGE jobs.

How should the changes be applied?

A Append every event to a table, and run MERGE every five minutes

B Use legacy streaming inserts, with the tariff ID as the insertId

C Stream rows tagged UPSERT or DELETE into a table with a primary key

D Use Datastream to replicate the pricing engine straight into BigQuery

◆ PAUSE AND DECIDE

How should the changes be applied?

A **Append and MERGE**

Works, but it is exactly the MERGE job the team ruled out.

B **insertId as the key**

insertId only drops duplicates briefly. It never updates a row.

C **UPSERT and DELETE**

BigQuery applies the changes itself, keyed on the primary key.

D **Datastream**

Datastream reads databases. This engine has none to read.

THE SIGNAL

Change events, no database, no MERGE jobs: CDC through the Storage Write API.

> Westgrid's daily settlement files sit in Cloud Storage as CSV, behind an external table. Larkspur's dashboards query them dozens of times an hour.
>
> The queries are slow and expensive, and a file never changes once it has arrived.

What should the team do?

A. Keep the external table, and buy more slots for the dashboards

B. Use federated queries with EXTERNAL_QUERY to read the files

C. Stream every row in again each hour, using the REST streaming API

D. Load each file into a native, partitioned table when it arrives

◆ PAUSE AND DECIDE

What should the team do?

A More slots

Pays more to scan the same CSV files again and again.

B EXTERNAL_QUERY

That reaches Cloud SQL, Spanner and AlloyDB, not files.

C Re-stream every hour

Pays to ingest the same rows repeatedly, and creates duplicates.

D Load once, natively

Files never change, so one free load gives fast, cheap queries.

THE SIGNAL

Queried all day, never changing: load once into native, partitioned tables.

> Larkspur's raw.meter_readings contains resent readings. Copies share meter_id and reading_ts, but each has its own received_ts.
>
> curated.readings_hh must hold exactly one row per meter per half hour, keeping the copy received last.

Which query logic is right?

A QUALIFY ROW_NUMBER, partitioned by meter and time, by received_ts

B SELECT DISTINCT across every column of the raw readings table

C Rely on the insertId set by the pipeline when the rows were written

D GROUP BY meter_id, keeping the maximum kwh value for each meter

◆ PAUSE AND DECIDE

Which query logic is right?

A QUALIFY ROW_NUMBER

One row per business key, and the latest copy wins.

B SELECT DISTINCT

The copies differ in received_ts, so none are removed.

C insertId

The meter sent the reading twice. The transport cannot know that.

D GROUP BY meter_id

Collapses every half hour into one row per meter.

THE SIGNAL

One row per business key, latest copy wins: ROW_NUMBER over the key, ordered by received time.

> Every morning, Larkspur's finance team needs a summary table rebuilt from curated.bills. It is one SQL statement, with no other steps and no dependencies.
>
> The team wants the simplest option with nothing to deploy.

What should run the statement?

A A Dataflow batch pipeline that rebuilds the table every morning

B A scheduled query that writes the result to a destination table

C A Cloud Run function on a cron schedule that runs the bq tool

D A Datastream stream from curated.bills into the summary table

◆ PAUSE AND DECIDE

What should run the statement?

A. A Dataflow pipeline

A whole pipeline for one SQL statement.

B. A scheduled query

One statement, a timetable, a destination table, nothing to deploy.

C. A Cloud Run function

Code and a scheduler to maintain, for one query.

D. Datastream

Datastream reads operational databases, not BigQuery tables.

THE SIGNAL

One statement, no dependencies, nothing to deploy: a scheduled query.

Module 16 in five lines

- **Load jobs** are free in the shared pool. Use them for batches of files.
- **Storage Write API** for streaming. Committed stream with offsets for exactly once.
- Legacy streaming is now **Storage Write API (REST)**. insertId is best effort only.
- Changes: **Datastream**, **UPSERT and DELETE**, or append then **MERGE**.
- Deduplicate on the **business key** with ROW_NUMBER. Load data that is queried all day.

COMING UP

Next: module 17, low-code, SQL pipelines and AI enrichment.